

Code-Layout, Readability and Reusability

Date	30 September 2026
Team ID	SWTID-2026-9357
Project Name	ComicCraftAI
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard 4-space indentation used
2	Proper File Structure	Files and folders are logically organized	Yes	Component and utility folders separated
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive naming conventions followed
4	Function / Method Names	Functions are descriptively named	Yes	CamelCase / PascalCase used consistently
5	Code Comments	Inline and block comments explain logic	Yes	JSDoc and inline comments added
6	Modular Design	Code is split into reusable functions/modules	Yes	High modularity achieved
7	No Redundant Code	Duplicate or unused code is removed	Yes	Cleaned up unused imports and dead code
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-catch blocks and error boundaries implemented

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	ComicPanelRenderer	React / TypeScript	Comic Creation & Preview Pages	High

2	AuthModule	Node.js, Express, JWT	Implemented as a reusable middleware.
3	UserProfileAPI	React, Redux, Axios	API client with clean data mapping.
4	FormValidationSvc	TypeScript, Joi	Modular library with clear error messages.
5	DatabaseConnPool	Python, SQLAlchemy	Shared connection pool with context managers.

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clean directory structure, consistent indentation, and clear modularization. No complex nesting.
Readability	5	Descriptive naming conventions for variables and functions. Logical code flow. Excellent use of whitespace.
Reusability	4	Most core components are modular and could be extracted into libraries. Some minor tight coupling remains in edge cases.
Documentation / Comments	5	comprehensive inline comments and API documentation. Clear README. Standard header comments.
Overall Score	4.75	Highly professional. Exceeds expectations in readability and structure. Minor refactoring for perfect reusability possible.